

Anwendungsorientierte Softwareentwicklung

Laborblatt 7: Bewegte Bilder

Lernziele

- Vertiefung objektorientierter Programmierung
- Refactoring bestehender Programme
- Inkrementellen Softwareentwicklung: kleine Änderungen, testen und versionieren

Schritt 1: Eigenes Repository klonen

Heute arbeiten wir wieder „nur“ mit Python. Ziel ist es, den bestehenden Code weiter zu verbessern (Stichwort *Refactoring*) und anschließend die Funktionalität auszubauen. Bevor wir mit Änderungen starten, verschafft euch bitte noch einmal einen Überblick über den aktuellen Stand des Spiels.

Wie immer: Öffnet zwei Terminals – eines zum Editieren (links), eines zum Ausführen (rechts).

Linkes Terminal (Editieren)

Repository klonen und ins Arbeitsverzeichnis wechseln:

```
git clone https://gitlab.uni-ulm.de/BENUTZERNAME/asp-ulm
cd asp-ulm/python
```

1. Aufgabe:

Öffnet die Datei `breakout.py` in eurem Editor und untersucht den Aufbau des Programms. Konzentriert euch dabei auf folgende Punkte:

- Geht in der Gruppe gemeinsam den Ablauf des Programms durch, insbesondere die Hauptschleife am Ende der Datei.
- Verschafft euch einen Überblick über die vorhandenen Klassen und deren Aufgaben.
- Sucht gezielt nach globalen Variablen (z. B. `screen`, `width`, `height`, `balls`, `running`) und stellt fest, wo sie verwendet oder verändert werden.
- Unterscheidet, von welchen Klassen Instanzen erzeugt werden (`Ball`) und welche nur als Klassenobjekt mit statischen Attributen und Methoden verwendet werden (`Paddle`).
- Rekapituliert dabei die Rolle von `__init__` und `self` in Klassen mit Instanzen.

Sprecht euch ab, bis alle den grundlegenden Aufbau verstanden haben. Diese Vorbereitung ist wichtig für das Refactoring im nächsten Schritt.

Rechtes Terminal (Kommandos ausführen)

Wechselt ebenfalls ins Python-Verzeichnis:

```
cd asp-ulm/python
```

2. Aufgabe

Führt das Spiel dem Befehl `python breakout.py` aus.

Vergleicht das beobachtete Verhalten mit dem, was ihr in Aufgabe 1 über den Code herausgefunden habt. Wenn es Abweichungen oder Unklarheiten gibt, besprecht diese mit uns, bevor ihr weitermacht.

Schritt 2: Refactoring

Die bisherigen globalen Variablen werden in eine eigene Klasse Game ausgelagert. Diese Klasse kommt in eine neue Datei `game.py`. Auch alle anderen Klassen werden ab jetzt jeweils in eine eigene Datei verschoben, damit jede Datei genau eine Funktionalität enthält und der Code übersichtlich bleibt.

Ziel ist:

- bessere Struktur
- weniger globale Variablen **außerhalb von Klassen**
- Vorbereitung auf weitere objektorientierte Schritte

Am Ende soll die Datei `breakout.py` fast nur noch die Hauptschleife enthalten.

Aufgabe 1

Erstellt die Datei `game.py` mit folgendem Inhalt:

```

1 import pygame
2
3 class Game:
4     width, height = None, None
5     screen = None
6
7     def init():
8         Game.width, Game.height = 800, 600
9         pygame.init()
10        Game.screen = pygame.display.set_mode((Game.width, Game.height))
11
12    def quit():
13        pygame.quit()

```

Aufgabe 2: `ball.py` (Referenz für die Datei am Ende der Aufgabe)

Erzeugt eine neue Datei `ball.py` durch Kopieren: `cp breakout.py ball.py`

Bearbeitet die neue Datei `ball.py` wie folgt:

- Löscht alles außer der Klasse `Ball`.
Tipp für **Nano**: Mit gehaltener **Shift-Taste** und den Pfeiltasten könnt ihr Textbereiche markieren. Mit **Control+K** lassen sich markierte Bereiche löschen.
- Ergänzt am Anfang die nötigen `import`-Zeilen:

```

1 from game import Game
2 import pygame

```

Hinweis:

Mit `from game import Game` wird direkt die Klasse `Game` aus dem Modul `game.py` importiert. Dadurch kann im Code einfach `Game.width` oder `Game.screen` geschrieben werden. Würde man stattdessen `import game` verwenden, müsste man jedes Mal den Modulnamen mit angeben, also z.B. `game.Game.width`.

Beide Varianten sind korrekt – aber wir wählen hier `from ... import ...`, damit der Code übersichtlicher bleibt.

- Passt den Zugriff auf globale Variablen an: Statt `width`, `height` oder `screen` verwendet ihr jetzt `Game.width`, `Game.height`, `Game.screen`.

Referenz: ball.py

```
1 from game import Game
2 import pygame
3
4 class Ball:
5     def __init__(self, x, y, vx, vy):
6         self.x = x
7         self.y = y
8         self.vx = vx
9         self.vy = vy
10        self.c = (255, 255, 0)
11
12    def move(self):
13        self.x += self.vx
14        self.y += self.vy
15        if self.x < 0 or self.x > Game.width:
16            self.vx *= -1
17        if self.y < 0 or self.y > Game.height:
18            self.vy *= -1
19
20    def draw(self, surface):
21        pygame.draw.circle(surface, self.c, (self.x, self.y), 5)
```

Aufgabe 2: paddle.py

Lagert die Klasse Paddle in eine eigene Datei `paddle.py` aus – analog zu `ball.py`.

Achtet darauf, dass auch hier der Zugriff auf globale Variablen über die Klasse Game erfolgt.

Die Initialisierung der Position erfolgt jetzt nicht mehr direkt bei der Definition, sondern durch einen Aufruf von `Paddle.init()`.

Referenz: paddle.py

```
1 from game import Game
2 import pygame
3
4 class Paddle:
5     c = (0, 0, 255)
6     w, h = 100, 30
7     x, y = None, None
8
9     def init():
10        Paddle.x, Paddle.y = (Game.width - Paddle.w) // 2, 570
11
12    def left():
13        Paddle.x -= 10
14        if Paddle.x < 0:
15            Paddle.x = 0
16
17    def right():
18        Paddle.x += 10
19        if Paddle.x + Paddle.w > Game.width:
20            Paddle.x = Game.width - Paddle.w
21
22    def draw(surface):
23        pygame.draw.rect(surface, Paddle.c,
24                          (Paddle.x, Paddle.y, Paddle.w, Paddle.h))
```

Aufgabe 3

Räumt die Datei `breakout.py` auf, sodass dort nur noch der Spielablauf in der Hauptschleife steht. Importiert die benötigten Klassen aus den Dateien `game.py`, `ball.py` und `paddle.py`. Passt wieder den Zugriff auf globale Variablen an. Initialisiert zu Beginn das Spiel mit `Game.init()` und ruft anschließend `Paddle.init()` auf.

Referenz: `breakout.py`

```

1 from game import Game
2 from ball import Ball
3 from paddle import Paddle
4 import time
5 import pygame
6
7 Game.init()
8 Paddle.init()
9
10 running = True
11 balls = [Ball(400, 300, -1, 2), Ball(200, 10, 4, 2)]
12 while running:
13     for event in pygame.event.get():
14         if event.type == pygame.QUIT:
15             running = False
16         elif event.type == pygame.KEYDOWN:
17             if event.key == pygame.K_q:
18                 running = False
19     keys = pygame.key.get_pressed()
20     if keys[pygame.K_a]:
21         balls.append(Ball(400, 300, -1, 2))
22     if keys[pygame.K_LEFT]:
23         Paddle.left()
24     if keys[pygame.K_RIGHT]:
25         Paddle.right()
26
27     pygame.Surface.fill(Game.screen, (0, 0, 0))
28     for i in range(0, len(balls)):
29         balls[i].draw(Game.screen)
30         balls[i].move()
31     Paddle.draw(Game.screen)
32     pygame.display.flip()
33     time.sleep(1/30)
34
35 Game.quit()

```

Aufgabe 4: Funktionstest, git commit und push

Startet das Spiel wie gewohnt mit `python breakout.py`. Prüft, ob das Spiel wie vorher funktioniert. Bewegt dazu mindestens einmal das Paddle ganz nach links und ganz nach rechts. Dadurch werden alle relevanten Bedingungen in `Paddle.left()` und `Paddle.right()` durchlaufen. Falls ihr beim Refactoring z. B. vergessen habt, `width` durch `Game.width` zu ersetzen, fällt das spätestens jetzt auf.

Dieser Test ist natürlich nicht vollständig – aber er deckt typische Fehler ab, die bei der Umstrukturierung passieren können.

Wenn das Spiel wie erwartet funktioniert, fügt die neuen Dateien dem Repository hinzu und macht einen Commit:

```

git add ball.py paddle.py
git commit -a -m "ein File macht ein Ding"
git push

```

Damit ist die Umstrukturierung abgeschlossen und sauber versioniert.

Schritt 3: Schnittstellen vereinfachen

Die Methode `draw` der Klasse `Ball` (in `ball.py`) sieht aktuell so aus:

```
# ...
class Ball:
    # ...

    def draw(self, surface):
        pygame.draw.circle(surface, self.c, (self.x, self.y), 5)
```

Aufgerufen wird sie nur an einer Stelle – in der Hauptschleife von `breakout.py`:

```
balls[i].draw(Game.screen)
```

Gezeichnet wird also immer auf `Game.screen`. Das ergibt Sinn – wir haben genau ein **Spiel** und eine **Zeichenfläche**.

Die Übergabe der Zeichenfläche ist damit überflüssig und macht den Code unnötig allgemein.

Unser Code wird noch früh genug komplex. Deshalb: Alles raus, was jetzt noch nicht gebraucht wird.

Aufgabe:

1. Vereinfacht die Methode `draw` in der Klasse `Ball` wie folgt:

```
# ...
class Ball:
    # ...

    def draw(self):
        pygame.draw.circle(surface, self.c, (self.x, self.y), 5)
```

Achtet darauf, dass ihr in der Methode `draw` jetzt `Game.screen` direkt verwendet.

Passt auch den Aufruf in `breakout.py` entsprechend an: `balls[i].draw()`

2. **Transferleistung:**

Auch die Methode `draw` der Klasse `Paddle` enthält dieselbe unnötige Flexibilität.

Passt sie ebenfalls so an, dass sie direkt `Game.screen` verwendet – und anschließend in `breakout.py` einfach mit `Paddle.draw()` aufgerufen werden kann.

3. Wenn alles wie erwartet funktioniert, committed eure Änderungen:

```
git commit -a -m "draw methoden vereinfacht"
git push
```

Schritt 4: Reflexion des Ball an den Wänden

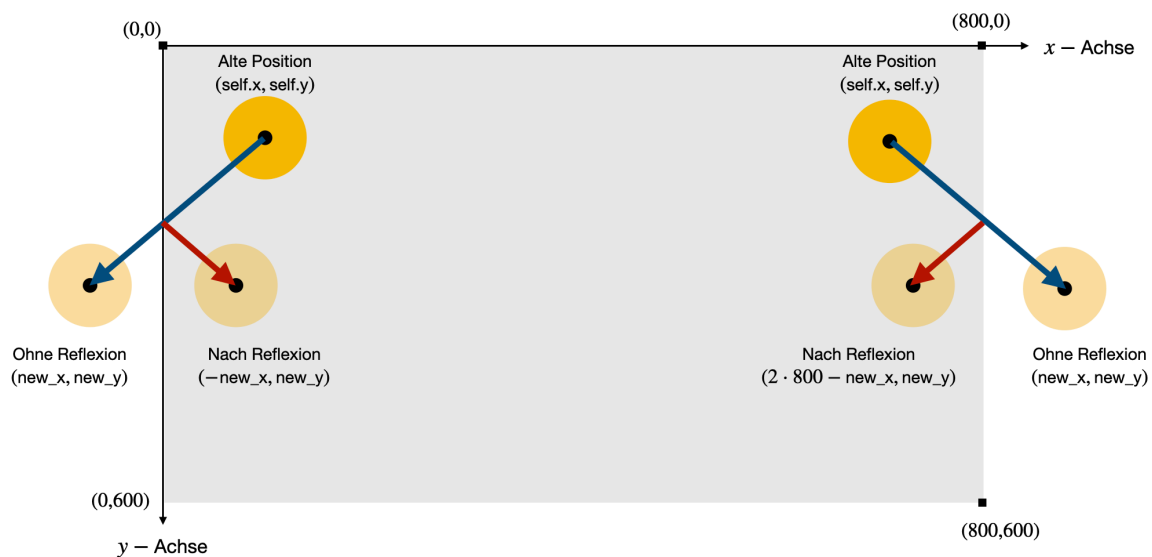
Es ist höchste Zeit, eine Skizze mit Koordinaten unseres Spielfelds zu zeigen:

Wie ihr vielleicht schon bemerkt habt (oder euch jetzt auffällt), liegt der Ursprung **links oben**, und die **y-Achse zeigt nach unten** – so ist es bei Pygame üblich.

In der folgenden Skizze sind zwei Bälle dargestellt, die sich jeweils auf eine Wand zubewegen. Ohne Reflexion würden sie im nächsten Frame die Wand durchdringen. Die Reflexion verhindert das – durch Umkehr der Bewegungsrichtung entlang der jeweiligen Achse.

Die Skizze zeigt:

- den aktuellen Mittelpunkt des Balls bei `(self.x, self.y)`
- die neue, unreflektierte Position `(new_x, new_y)`
- den zugehörigen Geschwindigkeitsvektor `(vx, vy)`
- den Aufprall auf die linke oder rechte Wand und die entsprechende Reflexion



Passt die `move`-Methode in der Klasse `Ball` (in `ball.py`) wie folgt an:

```

13     def move(self):
14         new_x = self.x + self.vx
15         new_y = self.y + self.vy
16
17         if new_x < 0:
18             new_x = -new_x
19             self.vx = -self.vx
20         if new_x > Game.width:
21             new_x = 2 * Game.width - new_x
22             self.vx = -self.vx
23         if new_y < 0:
24
25             self.vy = -self.vy
26         if new_y > Game.height:
27
28             self.vy = -self.vy
29
30         self.x = new_x
31         self.y = new_y

```

Aufgaben

1. Startet das Spiel mit `python breakout.py` und beobachtet, wie sich die Reflexion des Balls an der linken und rechten Wand verändert hat.

Ihr solltet sehen, dass der Ball (genauer: sein Mittelpunkt) realistisch reflektiert wird. Auch bei hoher Geschwindigkeit bleibt er immer **innerhalb der Zeichenfläche** – sofern wir den Ball als **Punktmasse** betrachten. Die Kugel selbst kann also zur Hälfte in die Wand hineinragen.

Wie kommt man auf den Code für die Reflexion?

- (A) Ist `new_x < 0`, dann ist der Ball um `-new_x` in die linke Wand eingedrungen. Bei einer Reflexion liegt die neue x-Koordinate symmetrisch zu `0` bei `-new_x`
- (B) Ist `new_x > Game.width`, dann ist der Ball in x-Richtung um `new_x - Game.width` über den rechten Rand hinausgelaufen. Die reflektierte x-Koordinate liegt symmetrisch zu `Game.width` bei `Game.width - (new_x - Game.width) = 2 * Game.width - new_x`

Hinweis wie man die Reflexion verbessern kann (freiwillig):

Wenn man es ganz genau machen möchte, müsste der Ball bei

3. `x = 5` (**linke Wand**)
4. `x = Game.width - 5` (**rechte Wand**)

abprallen – vorausgesetzt, der Radius des Balls ist 5.

Ihr solltet außerdem beobachten, dass der Ball **oben und unten** noch **aus dem Bildschirm herauslaufen kann** – das beheben als nächstes.

2. **Transferleistung: Reflexion an Boden und Decke**

Passt den Code in **Zeile 24** und **Zeile 26** so an, dass der Ball auch oben und unten realistisch reflektiert wird – analog zu den Seitenwänden.

3. Wenn das Spiel korrekt funktioniert und der Ball jetzt auch an Decke und Boden reflektiert, committed eure Änderungen:

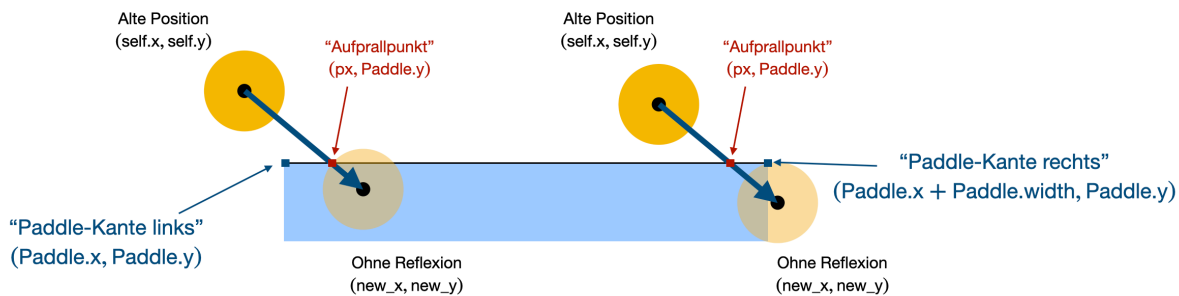
```
git commit -a -m "bessere Reflexion"
git push
```

Schritt 5: Reflexion am Paddle

Die folgende Skizze zeigt zwei Situationen, in denen ein Ball auf das Paddle treffen kann.

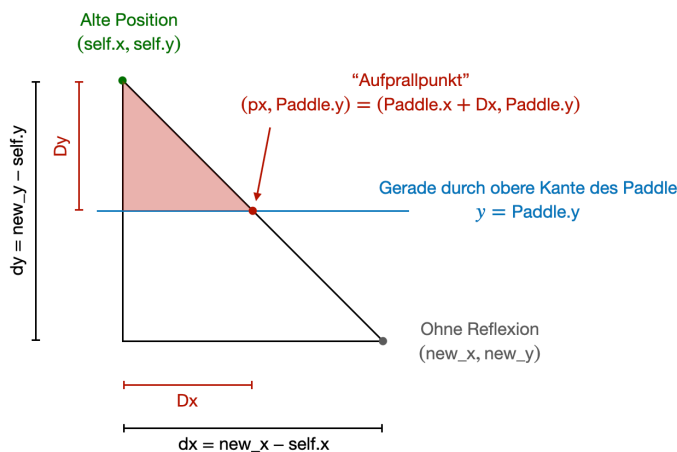
Gezeigt sind jeweils die aktuelle Position (`self.x`, `self.y`) und die berechnete neue Position (`new_x`, `new_y`), wenn der Ball **ungehindert weiterfliegt** – also **ohne Kollision**, wie es im aktuellen Code noch der Fall ist:

- Der Ball **links** befindet sich anfangs links neben dem Paddle. Sein Bewegungsvektor durchquert die obere Kante des Paddle, und die neue Position liegt **unterhalb der Kante und innerhalb des Paddle**.
- Der Ball **rechts** befindet sich zunächst **oberhalb** des Paddle. Seine neue Position liegt **rechts neben dem Paddle**, aber ebenfalls **unterhalb der oberen Kante**.



In beiden Fällen gilt: Der **Bewegungsvektor schneidet die obere Kante des Paddle**. An diesem **Aufprallpunkt** sollte die Reflexion stattfinden.

Um diesen Schnittpunkt möglichst einfach und allgemein berechnen zu können, betrachten wir folgende Skizze:



In der Skizze erkennt man zwei ähnliche Dreiecke – ein idealer Fall für den **guten alten Strahlensatz**. Der ist einfach, anschaulich und führt direkt zur Lösung – und trotzdem wird er im Studium oft vergessen oder durch unnötig komplizierte Rechnungen ersetzt. Hier zeigt sich, wie hilfreich solide Schulgeometrie auch in der Programmierung sein kann.

Unser Ziel: Wir wollen die **x-Koordinate des Aufprallpunkts** berechnen – also den Punkt, an dem der Bewegungsvektor des Balls die obere Kante des Paddle schneidet. Dazu setzen wir die Streckenverhältnisse der beiden Dreiecke ins Verhältnis:

$$\frac{Dx}{Dy} = \frac{dx}{dy} \text{ mit } dx = new_x - self.x \text{ und } dy = new_y - self.y$$

Die vertikale Strecke bis zur Paddle-Kante ist:

$$Dy = \text{Paddle.y} - \text{self.y}$$

Dann lässt sich der horizontale Anteil berechnen mit:

$$Dx = \frac{dx}{dy} Dy$$

Die x-Koordinate des Aufprallpunkts lautet schließlich:

$$px = \text{self.x} + Dx$$

Aufgaben

1. Das folgende Code-Fragment prüft, ob der Ball die **Gerade durch die obere Kante des Paddles** überquert. Ist das der Fall, soll der **Schnittpunkt** mit dieser Kante berechnet werden (diese Lücke im Code füllt ihr selbst). Danach wird geprüft, ob der Schnittpunkt innerhalb der **horizontalen Ausdehnung des Paddles** liegt. Falls ja, erfolgt die Reflexion durch Umkehrung der vertikalen Geschwindigkeit.

```
if self.y < Paddle.y and new_y >= Paddle.y:
    # In die dieser Lücke px berechnen
    if px >= Paddle.x and px <= Paddle.x + Paddle.w:
        new_y = Paddle.y + Dy - dy
        self.vy = -self.vy
```

Transferleistung: Überlegt euch, wo im Code dieses Fragment sinnvoll eingebaut werden sollte, um den Ball beim Auftreffen auf das Paddle zu reflektieren.

Hinweis für Stilbewusste: Verwendet bei der Division eine **Ganzzahldivision** (`//`), wo immer es sinnvoll ist. Auch wenn es in diesem kleinen Spiel keine Rolle spielt – im Herzen jedes HPC-Programmierers schmerzt unnötige Fließkommadivision ein bisschen.

2. Wenn der Ball jetzt wie gewünscht am Paddle reflektiert wird, committed eure Änderungen:

```
git commit -a -m "Paddle tut was es soll"
git push
```

Schritt 6: Breakout typisches Abprallen am Paddle

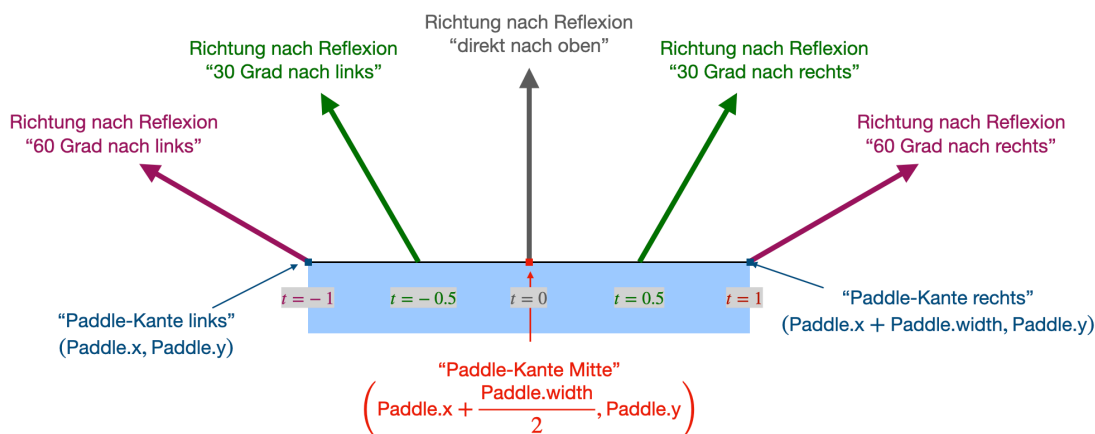
In klassischen Breakout-Spielen wird der Ball beim Aufprall auf das Paddle **nicht einfach reflektiert**. Stattdessen hängt seine neue Bewegungsrichtung davon ab, **wo auf dem Paddle** der Ball auftrifft.

Ein einfaches Modell:

- Trifft der Ball **genau in der Mitte** des Paddles, fliegt er **senkrecht nach oben**.
- Trifft er an der **linken Kante**, fliegt er **nach links oben**.
- Trifft er an der **rechten Kante**, fliegt er **nach rechts oben**.

Der Ball soll dabei seine bisherige **Geschwindigkeit (Betrag)** beibehalten. Nur die **Richtung** ändert sich – und zwar **linear abhängig vom Aufprallpunkt**.

Die folgende Skizze veranschaulicht diese Idee:



Wir beschreiben den **Aufprallpunkt** auf dem Paddle durch einen Parameter $t \in [-1, 1]$, sodass:

- $t = -1$: "ganz links",
- $t = 0$: "genau in der Mitte",
- $t = 1$: "ganz rechts" entspricht.

Dazu parametrisieren wir die x-Koordinate des Aufprallpunkts p_x mit

$$p_x = \text{Paddle.x} + \frac{\text{Paddle.w}}{2} \cdot (1 + t) \quad \text{mit } t \in [-1, 1]$$

Nach t aufgelöst ergibt sich:

$$t = (p_x - \text{Paddle.x}) \cdot \frac{2}{\text{Paddle.w}} - 1$$

Man kann nun den Parameter $t \in [-1, 1]$ auf einen Winkel φ im **Bogenmaß** abbilden. Dabei müssen ein paar lästige Details berücksichtigt werden – zum Beispiel, dass in unserem Koordinatensystem die **y-Achse nach unten** zeigt. Mit der folgenden Formel ist das bereits korrekt berücksichtigt:

$$\varphi = \left(\frac{t}{3} - 1 \right) \cdot \frac{\pi}{2} \in \left[-\frac{4\pi}{6}, -\frac{2\pi}{6} \right].$$

Das bedeutet

- $t = -1$ ergibt einen Winkel von **60° nach oben links**.
- $t = 1$ ergibt **60° nach oben rechts**.
- $t = 0$ ergibt **senkrecht nach oben**.

Der Betrag der Geschwindigkeit bleibt erhalten:

$$v = \sqrt{\text{self.vx}^2 + \text{self.vy}^2}$$

Der neue Geschwindigkeitsvektor ergibt sich dann durch:

$$\text{self.vx} = v \cdot \cos \varphi \quad \text{und} \quad \text{self.vy} = v \cdot \sin \varphi.$$

Aufgaben

1. Jetzt wird die Theorie wieder in die Praxis umgesetzt!

Für die Umsetzung benötigen wir einige mathematische Funktionen, die im Modul `math` enthalten sind. Bindet das Modul in `ball.py` ein:

```
import math
```

Was wir daraus brauchen:

- `math.pi`: Konstante π
- `math.hypot(x, y)`: berechnet $\sqrt{x^2 + y^2}$
- `math.cos` und `math.sin` berechnen Cosinus und Sinus eines Winkels im **Bogenmaß**.

Mit diesen Hilfsmitteln sollte es nicht schwer sein, das folgende Code-Fragment zu vervollständigen und **an geeigneter Stelle** im Code zu platzieren:

```
if px >= Paddle.x and px <= Paddle.x + Paddle.w:
    new_y = Paddle.y + Dy - dy
    # In dieser Lücke t, phi, v, self.vx, self.vy berechnen
```

Bemerkung: Streng genommen reflektieren wir den Ball zuerst noch nach dem Prinzip „Einfallswinkel = Ausfallswinkel“ – und ändern danach erst die Richtung gemäß den **Breakout-Regeln**.

Das ist nicht ganz sauber, aber für das menschliche Auge wird es genauso aussehen, als ob wir direkt breakout-typisch reflektieren.

2. Wenn alles funktioniert, dann wie immer: Rein ins Repository!

```
git commit -a -m "Breakout-Reflexion implementiert"
git push
```

Schritt 7: Mal was ohne Theorie

Jetzt wird einfach nur der Code ein bisschen schöner gemacht – ganz ohne Theorie. Ersetzt in `breakout.py` diese Schleife:

```
for i in range(0, len(balls)):
    balls[i].draw()
    balls[i].move()
```

durch die kompaktere und besser lesbare Variante:

```
for b in balls:
    b.draw()
    b.move()
```

Beide Varianten machen exakt dasselbe – aber die zweite ist klarer, kürzer und dem Python-Stil entsprechend.

Wie immer: Testet, ob noch alles wie erwartet funktioniert. Dann ab damit ins Repository mit

```
git commit -a -m "Schönere Schleife"
git push
```

Schritt 8: Noch schnell ein neues Feature (Raketen!)

Wir haben jetzt viel Zeit damit verbracht, unseren Code aufzuräumen, schöner zu strukturieren und Details wie die Ballreflexion sauber zu lösen. Höchste Zeit zu überprüfen, ob sich das Ganze auch in der Praxis auszahlt!

Unser Paddle bekommt jetzt eine neue Fähigkeit: **Raketen abfeuern!**

Die Raketen sollen nach oben fliegen, also ähnlich wie Bälle, nur in umgekehrter Richtung. Zunächst werden sie sogar genau wie Bälle dargestellt – das ändern wir vielleicht später.

Erstellt eine neue Datei `rocket.py`. Der einfachste Weg ist wahrscheinlich, die Datei `ball.py` zu kopieren und anzupassen: `cp ball.py rocket.py` Ersetzt den Inhalt dann durch:

```
from game import Game
from paddle import Paddle
import pygame
import math

class Rocket:
    def __init__(self, x, y, vy):
        self.x = x
        self.y = y
        self.vy = vy
        self.c = (255, 255, 255)

    def move(self):
        if self.y < 10:
            return
        self.y += self.vy

    def draw(self):
        pygame.draw.circle(Game.screen, self.c, (self.x, self.y), 5)
```

Aufgaben

Folgende Änderungen in `breakout.py` genügen:

1. Import der Raketenklasse

Ganz oben bei den Imports (am besten bei den anderen `from ... import ...` Zeilen), füge hinzu:

```
from rocket import Rocket
```

Damit kannst du später direkt `Rocket(...)` schreiben, ohne `rocket.Rocket(...)`.

2. Liste der Raketen anlegen

In der Nähe, wo auch die Bälle initialisiert werden (z. B. direkt danach), füge hinzu:

```
rockets = []
```

Anfangs ist alles friedlich – noch keine Raketen unterwegs.

3. Pfeiltaste ↑ abfangen und Rakete erzeugen

Erweitere die Eventbehandlung für Tastendrucke (du prüfst dort schon `pygame.KEYDOWN` und `event.key == pygame.K_q`) wie folgt:

```
elif event.type == pygame.KEYDOWN:
    if event.key == pygame.K_q:
        running = False
    elif event.key == pygame.K_UP:
        rockets.append(Rocket(Paddle.x + Paddle.w//2, Paddle.y, -10))
```

Die Startposition ist die Mitte des Paddles, die Geschwindigkeit ist -10 (weil bei uns die y-Achse nach unten zeigt).

4. Raketen bewegen und zeichnen

In der Hauptschleife werden bisher alle Bälle bewegt und gezeichnet – jetzt machst du das Gleiche auch für die Raketen. Direkt danach:

```
for r in rockets:
    r.draw()
    r.move()
```

Ab jetzt kann man das Feature testen mit `python breakout.py`.

5. Wenn das alles läuft und Raketen fliegen – rein ins Repository:

```
git add breakout.py rocket.py
git commit -m "Raketen eingebaut"
git push
```

Schritt 9: Zum Abschluss räumen wir auf!

Unsere Raketen bleiben an der Decke kleben – und zwar mit Absicht!

In der Methode `move` wurde bisher nur geprüft, ob `self.y < 10` ist, und dann einfach **nichts mehr gemacht**. Das sorgt dafür, dass sie genau dort stehen bleiben.

Der Wert `10` wurde **absichtlich** statt `0` gewählt – so sieht man den „Müll“, den unser Code bisher produziert.

Aufgabe

Jetzt räumen wir auf: Die Klasse `Rocket` bekommt ein neues Attribut `alive`. Wenn eine Rakete die Decke erreicht (also `y < 10`), dann wird dieses Attribut auf `False` gesetzt – und wir entfernen sie dann aus der Liste `rockets`.

Neue Klasse `Rocket` in `rocket.py`:

```
class Rocket:
    def __init__(self, x, y, vy):
        self.x = x
        self.y = y
        self.vy = vy
        self.c = (255, 255, 255)
        self.alive = True

    def move(self):
        if self.y < 10:
            self.alive = False
            return
        self.y += self.vy

    def draw(self):
        pygame.draw.circle(Game.screen, self.c, (self.x, self.y), 5)
```

In der Hauptschleife von `breakout.py`

füge nach dem Zeichnen und Bewegen der Raketen folgende Zeile ein:

```
rockets = [item for item in rockets if item.alive]
```

Was macht diese Zeile?

Das ist eine sogenannte **Listenkompensation**. Sie erstellt eine neue Liste, die **nur die Raketen enthält**, bei denen `alive == True` ist – also solche, die noch nicht an der Decke „explodiert“ sind. **Kurz:** „Behalte nur die lebendigen Raketen.“

Wenn alles klappt, könnt ihr die `move`-Methode einer Rakete noch etwas verbessern: Setzt `alive` auf `False`, sobald die y-Koordinate kleiner als `0` ist.

Achtung: Danach unbedingt noch einmal testen, ob alles weiterhin korrekt funktioniert – um typische Last-Minute-Tippfehler auszuschließen.

Dann rein ins Repository:

```
git add rocket.py breakout.py
git commit -m "Raketen verschwinden an der Decke"
git push
```

Abschlussbemerkung:

Mit dem gleichen Vorgehen kann man auch Bälle verschwinden lassen, die man nicht mit dem Paddle trifft. Damit werden wir beim nächsten Mal beginnen – um direkt an das heutige Ende anzuknüpfen.